

AI 協作開發報告

UML Editor 新功能實作 — Claude AI 協作歷程紀錄

課程：物件導向程式設計

專案：Java Swing UML Editor (CodeModify)

AI 工具：Claude Sonnet 4.6 (透過 Claude Code CLI)

日期：2026 年 5 月

新增功能：虛線箭頭 · 紅點 Port 提示 · 拖曳建立物件 · Port 點擊高亮連線

1. AI 協作流程與策略

1.1 專案背景與任務說明

本次作業目標是在既有的 Java Swing UML Editor 上新增四項功能，同時**嚴格禁止重構整個專案架構**（違反者以零分計）。專案使用 Handler / Listener / Model 三層架構，核心檔案包含 `CanvasPanelHandler`、`FuncPanelHandler`、`CPHActionListener` 以及各 `mod.instance.*` 元件類別。

四項新功能需求如下：

#	功能	說明
1	Dependency Line (虛線箭頭)	新增虛線箭頭連線類型，以 <code>BasicStroke</code> 虛線繪製
2	Port 紅點提示	在連線模式下，滑鼠移到物件附近時顯示紅色圓點
3	拖曳建立物件	在 <code>Class / UseCase</code> 模式下，拖曳決定物件大小
4	Port 點擊高亮連線	在 <code>Select</code> 模式下，點擊 Port 將相連的線條高亮為藍色

1.2 協作流程概覽



Phase 1 — 需求理解與規格確認

上傳規格文件，要求 AI 閱讀並記憶所有需求與限制，不急於實作。



Phase 2 — 規格文件與 Task 生成

要求 AI 產出 spec 與 task 清單，形成可追蹤的實作計劃。



Phase 3 — 逐步實作

按 task 逐步進行，AI 修改既有檔案並新增 `DependencyLine.java`。



Phase 4 — 執行驗證 (截圖回饋循環)

執行程式後截圖給 AI，描述觀察到的問題，AI 分析並修正。共進行 4 輪。

Phase 5 — 連線邏輯深度除錯

針對連線困難問題，AI 分析 Swing 事件分派機制根本原因並修正。

1.3 關鍵 Prompt 與對應回覆

Prompt 1 — 需求閱讀階段

USER PROMPT

請你詳細閱讀此份文件，內容完本次要完成的新功能規格，請你先閱讀完，了解了所有的需求和規範之後再開始規劃。目前先幫我閱讀並記憶你所該遵守 即可能規畫如何進行。

AI 回覆策略

AI 閱讀規格文件後，拆解出四個功能需求，並特別記錄了禁止重構架構的限制條件。確認了 FuncIndex 對應關係 (0=Select, 1=Association ... 6=DependencyLine) 以及 AreaDefine 常數 (TOP=3, RIGHT=2, LEFT=1, BOTTOM=0) 等關鍵細節後，才進入下一階段。

Prompt 2 — 規格與 Task 文件生成

USER PROMPT

幫我建立 spec 相關檔案，產生 specify 等等 md 檔，後續再生成應該要有的 task，最後再進行實作。

AI 回覆策略

AI 建立了 **SPEC.md** 與 **TASKS.md** 兩份文件，將四個功能拆解為具體的可執行 task (含目標檔案、新增方法、修改邏輯)，並以此作為後續實作的錨點，確保每個步驟有明確的驗收標準。

Prompt 3 — 逐步實作

USER PROMPT

幫我按照 task 逐步實作。

AI 實作內容摘要

- 新建 `DependencyLine.java` (虛線 + 開口三角箭頭)
- 在 `FuncPanelHandler` 新增 `makeDependencyIcon()` 、 `getIcon(6)` 、 `getFunc(6)`
- 在 `Core.java` 新增 `isDependencyLine()` 並更新 `isLine()` / `isFuncComponent()`
- 在 `CanvasPanelHandler` 新增紅點繪製、拖曳建立物件、Port 高亮邏輯
- 修改 `CPHActionListener` 新增 `wasDragged` 旗標
- 在 `AssociationLine` 、 `CompositionLine` 、 `GeneralizationLine` 新增 `highlight` 機制

Prompt 4 — 第一輪截圖除錯

USER PROMPT (附截圖)

目前好像沒有虛線箭頭，都是實線的，且圖片會破圖，連線也很難連到，切換模式也常常失敗。幫我看一下要如何改，記得不要重購專案。

AI 診斷與修正

AI 發現虛線箭頭的圖示 icon 因使用 `dependency_line.jpg` (實際上是 Association 圖) 而顯示錯誤。修正方案：改為在 `FuncPanelHandler` 用程式碼以 `BufferedImage + Graphics2D + BasicStroke` 動態繪製虛線圖示，不依賴外部圖檔。

Prompt 5 — 第二輪截圖除錯

USER PROMPT (附截圖)

目前沒辦法用拖曳大小的方式去建立 UseCase 跟 Class，且連線部分常常失敗，滑鼠移過去也不會顯示紅點，不太好操作，幫我修改一下。

AI 診斷與修正

發現兩個問題：(1) Swing 的 `MouseMotionListener` 不會從子元件往父元件傳遞，因此移到物件上時紅點沒有觸發。修正：在每個 `addObject` 時附加 `MouseMotionAdapter` 並用 `SwingUtilities.convertPoint()` 轉換座標。(2) `mouseClicked` 在拖曳後仍會觸發，造成雙重建立物件。修正：在 `CPHActionListener` 加入 `wasDragged` 旗標。

Prompt 6 — 第三輪截圖除錯

USER PROMPT (附截圖)

現在仍然無法用拖曳的方式建立物件，另外跟我講一下現在連接線段要怎麼連，另外還有一個問題是紅點目前好像不是在圖層最上面，會被遮住半個點，幫我改一下。

AI 診斷與修正

發現三個問題：(1) `BasicClass` 與 `UseCase` 的 `paintComponent` 仍使用 `defSize` 繪圖而非 `this.getWidth() / getHeight()`，導致自訂大小無效。(2) 紅點在 `paintComponent` 中繪製，會被子元件遮住。修正：改為 `override paintChildren`，在子元件繪製後才畫紅點。

Prompt 7 — 第四輪連線邏輯除錯

USER PROMPT (附截圖)

好像還是連不太起來欸，我都等到有紅點出現才放開了。另外現在有一個 `class` 只能連一條等等的規定嗎？`User Case` 可以連到其他的 `Use Case` 嗎？幫我統整目前的連線規則。

AI 根本原因分析

深入分析 Swing 事件分派機制後發現：當使用者在物件內部按下滑鼠時，Swing 的 `LightweightDispatcher` 將事件分派給子元件 (`BasicClass/UseCase`)，`contextPanel` 的 `CPHActionListener` 完全收不到 `mousePressed`，導致 `from` 點從未被設定，連線必然失敗。

修正：(1) 在 `CanvasPanelHandler` 儲存 `cphListener` 為欄位。(2) 在每個物件上附加 `MouseAdapter`，將 `pressed/released/clicked` 事件透過 `SwingUtilities.convertPoint()` 轉換後轉發給 `cphListener`。(3) 加入 `isInsideWithTolerance()` 方法 ($\pm 15\text{px}$ 容許誤差)，讓從物件邊緣出發的連線也能被正確識別。

2. 驗證歷程

2.1 驗證策略

本次協作採用「實作 → 編譯 → 執行 → 截圖回饋」的迭代循環，每一輪均以截圖作為驗證依據，讓 AI 能直接觀察程式執行結果。同時每次修改後 AI 會立即重新編譯確認無語法錯誤。

驗證方式	說明
編譯驗證	每次修改後執行 <code>javac</code> ，以零錯誤輸出確認語法正確
截圖回饋	執行程式後截圖並描述異常行為，AI 根據視覺資訊診斷
行為描述	描述「按了但沒有反應」、「紅點不顯示」等具體症狀
架構限制確認	明確要求 AI 不重構專案，每次修改後確認限制仍被遵守

2.2 Bug 發現與修正歷程

Bug 1 — 虛線箭頭 Icon 顯示錯誤

問題描述

虛線箭頭按鈕顯示與 Association Line 相同的圖示，且來自不存在的圖檔 `dependency_line.jpg`，導致圖示破圖。

修正方案

在 `FuncPanelHandler.makeDependencyIcon()` 中以程式繪製圖示：用 `BasicStroke` 虛線畫橫線，加上開口三角箭頭。不依賴外部圖檔。

Bug 2 — 拖曳建立物件後大小仍顯示預設值

問題描述

`setCustomSize()` 與 `reSize()` 雖已實作，但 `paintComponent()` 中繪製矩形/橢圓時仍寫死使用 `defSize`，導致物件外觀不隨拖曳大小改變。

修正方案

將 `BasicClass.paintComponent()` 和 `UseCase.paintComponent()` 中所有 `defSize.width / defSize.height` 改為 `this.getWidth() / this.getHeight()`。

```
// 修正前 (BasicClass)
g.fillRect(0, i * defSize.height, defSize.width - 1, defSize.height - 1);

// 修正後
int w = this.getWidth();
int rowH = texts.size() > 0 ? this.getHeight() / texts.size() :
defSize.height;
g.fillRect(0, i * rowH, w - 1, rowH - 1);
```

Bug 3 — 紅點被物件遮住

問題描述

紅點在 `contextPanel` 的 `paintComponent()` 中繪製，但 Swing 繪圖順序為 `paintComponent` → `paintChildren`，子元件（`BasicClass/UseCase`）繪製後會覆蓋紅點。

修正方案

將 `contextPanel` 的 anonymous `JPanel` override 方法從 `paintComponent` 改為 `paintChildren`，讓紅點在所有子元件繪製完畢後才畫，確保永遠在最上層。

```
// 修正前
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    drawPortHints(g); // 在子元件之前畫 → 被蓋住
}

// 修正後
protected void paintChildren(Graphics g) {
    super.paintChildren(g);
    drawPortHints(g); // 在子元件之後畫 → 永遠在最上層
}
```

Bug 4 — 從物件內部無法啟動連線拖曳

問題描述

這是最核心的 Bug。當使用者在 `BasicClass / UseCase` 內部按下滑鼠開始拖曳時，連線幾乎都失敗。原因是 Swing 的 `LightweightDispatcher` 事件分派機制：子元件（`BasicClass`）雖然沒有 `MouseListener`，但有 `MouseMotionListener`，Swing 因此將 `mouse press` 事件分派給子元件，`contextPanel` 的 `CPHActionListener` 完全收不到 `mousePressed`，導致 `from` 點從未被設定，連線必然失敗。

修正方案 (三層修正)

1. 在 `CanvasPanelHandler` 將 `CPHActionListener` 改存為欄位 `cphListener`
2. 在 `addObject()` 為每個新建物件加上 `MouseAdapter`，將事件轉換座標後轉發給 `cphListener`
3. 新增 `isInsideWithTolerance()` 方法 ($\pm 15\text{px}$ 容許誤差)

```
obj.addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        Point p = SwingUtilities.convertPoint(obj, e.getPoint(),
contextPanel);
        cphListener.mousePressed(new MouseEvent(
            contextPanel, e.getID(), e.getWhen(),
            e.getModifiersEx(), p.x, p.y, e.getClickCount(), false));
    }
    // mouseReleased / mouseClicked 同理
});
```

Bug 5 — mouseClicked 在拖曳後仍觸發導致重複建立物件

問題描述

Java Swing 的 `mouseClicked` 在滑鼠移動後仍然會觸發，導致拖曳建立物件後，`mouseClicked` 又呼叫一次 `addObject()`，造成重複建立。

修正方案

在 `CPHActionListener` 新增 `wasDragged` 旗標，在 `mouseReleased` 時根據位移量 ($> 5\text{px}$) 判斷，`mouseClicked` 中若 `wasDragged == true` 則跳過。

2.3 架構完整性確認

每次實作均在既有架構框架內進行，無重構行為：

- 新增類別：`DependencyLine.java` 依照既有的 `AssociationLine` 結構實作，實作相同介面

- **修改現有類別**：均為最小範圍修改（新增方法或修改特定方法），未改變既有方法簽名
- **分層原則**：Handler 層處理邏輯，Listener 層處理事件，mod.instance 層處理繪圖，三層分工未被破壞
- **編譯通過**：每次修改後均以 `javac` 確認零錯誤

3. 心得與反思

3.1 AI 協作的優勢

快速理解大型程式碼庫

傳統上要理解一個陌生 Java Swing 專案需要花大量時間閱讀程式碼。透過 AI，只需要讓它讀取關鍵檔案，它就能快速建立各元件之間的依賴關係圖，並在實作時精準定位需要修改的位置。例如 AI 主動識別出 `FuncPanelHandler.init()` 的迴圈靠 `getIcon(index) != null` 決定按鈕數量，因此只需要在 `getIcon(6)` 加入回傳值即可自動新增第七個按鈕，無需改動其他邏輯。

深度技術知識的即時援用

本次最困難的 Bug (Swing 事件不從子元件傳遞到父元件) 涉及到 Java AWT `LightweightDispatcher` 的底層機制。這類知識即使是有經驗的開發者也不一定熟悉。AI 能夠直接指出根本原因並給出解法 (`MouseAdapter` 轉發 + `SwingUtilities.convertPoint()`)，節省了可能需要數小時的除錯時間。

截圖作為溝通橋樑

在 UI 開發中，「描述問題」往往很困難。本次協作發現截圖是最有效的溝通方式，讓 AI 能直接觀察「紅點被遮住」、「物件大小看起來沒有改變」等視覺問題，大幅縮短了問題描述的成本。

3.2 AI 協作的限制與挑戰

需要人工執行驗證循環

AI 無法自行執行程式或看到程式執行結果。每次實作後必須由使用者手動執行、截圖、描述問題，再回饋給 AI。這個循環的效率取決於使用者描述問題的精確度——「連不太起來」這樣的描述需要多輪追問，而「從物件內部按下開始拖曳時連線失敗」則能讓 AI 更快定位問題。

上下文長度的限制

隨著對話越來越長，AI 的上下文會達到上限並被壓縮摘要。在跨對話的連續開發中，需要確保關鍵的技術決策和已知問題被明確記錄，否則 AI 可能在下一輪重新提出已解決的方案。本次使用了 MEMORY.md 持久記憶機制來緩解此問題。

架構限制需明確重申

「不重構」的限制必須在每次提問時明確重申。AI 有時會傾向提出更乾淨的重構方案，需要使用者主動提醒遵守既有架構約束。這也提醒我們：Prompt 中的限制條件和 Prompt 中的需求同樣重要。

3.3 策略建議

策略	說明
先讀後寫	先讓 AI 讀懂既有程式碼，再要求修改，避免 AI 憑空假設架構
小步迭代	每次只修改一個功能，立即編譯驗證，減少多個 Bug 同時出現的混亂
截圖優先	UI 問題優先使用截圖描述，比文字描述更直觀有效
明確限制	將「不能做什麼」（如不重構）明確寫在 Prompt 中，AI 才會遵守
追問根本原因	Bug 修不好時，追問「根本原因是什麼」比直接要求重試更有效

3.4 總結

本次 AI 協作開發經驗顯示，AI 在理解複雜程式庫、定位技術問題根因、以及生成符合既有架構的程式碼方面具有顯著優勢。然而，AI 無法自行執行程式，因此人機協作的核心仍然是：**人負責執行與觀察，AI 負責分析與實作**。

整個開發過程中，最有價值的一課是：*清晰描述問題比請求 AI 直接修正更重要*。當我能精確說明「在物件內部按下後 `contextPanel` 收不到 `mousePressed`」時，AI 能夠立即提出根本解法；而當只說「連線連不上」時，則需要多輪對話才能收斂到正確答案。

未來在使用 AI 協助開發時，建議投入時間在「問題表述的精確化」，這是提升 AI 協作效率的最關鍵因素。